LAT Pointing Requirements *

H. Nachman

University of Texas at Austin

Austin, Texas

October 11, 2026

1 Overview

The basic goal of this project is to determine what the pointing requirements are for various SO LAT science goals, and evaluate whether these goals have been achieved (and ultimately, if not, explore what elements of pointing are limiting us). Some of the framework for how this project will proceed are based on Elisa Russier's talk at the F2F 2026.

The goal can be thought of in a sentence such as:

> **X** must be known to within **a%** to avoid biasing **Y** by more than **b%**.

In the case of pointing, this might look like:

> Pointing must be known to within 5 arcsec to avoid biasing σ_8 by more than 2%. (these are random numbers just for example)

Where do each of these components come from?

*Correspondence to: henry.nachman@utexas.edu

1.1 Expected (simplified) Pipeline:

1. Simulate pointing errors in maps:
    - TOAST - TOD based simulation framework. Would allow me to consider pointing errors in maps, but might be more complex than needed.
    - SMARTIES - Map-space simulation, should allow me to consider pointing errors in maps - less documentation. Might also be limited to static pointing offsets which could be accounted for in pointing models.
2. Maps to power spectra:
    - Pass simulated maps through realistic pipelines to achieve realistic power spectra.
    - What different 'data products' (eg. types of power spectra) are needed to perform step 3.
3. Analyze power spectra to put constraints on parameters:
    - Different parameters might have different estimation pipelines.
    - Is this via MCMC parameter estimation techniques like CosmoMC/Cobaya + CAMB/CLASS that will give me confidence intervals for parameters given a single pointing realization?
    - Or more simply, once I have the parameter constraints – one with no pointing errors and one with pointing errors – I can compare the two to see how the addition of pointing errors impacts the product.

1.2 How to start / guidance I need:

1. Figure out how to simulate maps + include pointing errors (jitter beams) into maps:
    - If the path is TOAST - then I think I have what I need to at least get started with sims but it might take a bit of figuring out before I have maps that I can use.
    - If the path is SMARTIES - Who can I talk to about SMARTIES because I have not yet found much documentation. Using SMARTIES will require I have a better understanding of how exactly different sources of pointing errors impact maps

- **What pointing errors am I concerned with? Beam pointing offsets → simple 'beam smoothing'. Jitter beams and more complex situations?**
- **What sort of mapmaking simulation framework should I be using/starting on? And what resources are there (people/documentation/examples) to help facilitate me learning this framework?**
- **How realistic should my maps be? Do I start simple and ignore things like point sources, atmosphere models, noise? Presumably different science cases rely on different things, so different maps needed, point-sourced filtered, delensed?**

2. How do I determine the uncertainty in a parameter - this is very new to me?
   - **What does the parameter estimation (and uncertainty estimation) pipeline look like? How should I behind to approach this?**
   - **Which cosmological parameter should I focus on? Map science vs transient science?**
   - Do we know which case(s) have the strictest pointing needs?
   - Map science vs transient science? Does transient science analysis rely solely on the static pointing model or can it use point-source corrections?
   - How does one go from a map to a parameter constraint? CosmoMC/CLASS/CAMB with MCMC parameter estimation methods (as in ACT)?

1.3 Useful resources

- Elle Shaw F2F 2026 - T1a: LAT Calibration - Pointing Summary
- Zach Atkins + F2F 2026 - W1a: LAT Science Readiness - Power-spectrum pipelines
- Cristian Vargas F2F 2026 - W1a: LAT Science Readiness - Beam Jitter
- Magdy Morshed F2F 2026 - W2b: Simulations - SMARTIES
- Elisa Russier F2F 2026 - W2b: Simulations - Beam Systematics Requirements

1.4 Science Goals

The enhanced Large Aperture Telescope (LAT) has a number of science goals across cosmological, astrophysical, and particle physics fields, and stemming from a variety of different measurement/analysis channels. Understanding how pointing impacts the science results requires pairing each science goal with its respective analysis method. For example, **?** states the enhanced LAT will place a constraint on the number of relativistic species $\sigma(N_{\rm eff}) = 0.045$. This constraint is largely set by imprints in the small-scale damping tail of the CMB temperature and polarization anisotropy power spectra. Thus, to determine how pointing impacts this constraint on $N_{\rm eff}$, one would need to simulate how changes in pointing impact the power spectra (in this case we might predict that pointing errors that express themselves as smoothing of the beam would effective limit/increase uncertainty on the smallest scales of the power spectrum).

Table 1: Summary of Enhanced Science Goals from SO LAT Survey[a]

	Current[b]	**SO 2025–2034**	Using Rubin, DESI, or *Euclid*	Reference
Primordial perturbations				
n_s	0.004	0.002	-	Shandera 2019
$e^{-2\tau}\mathcal{P}(k = 0.2\,\mathrm{Mpc}^{-1})$	3%	0.4%	-	Slosar 2019
$f_{\rm NL}^{\rm local}$	5	1	✓	Meerburg 2019
Relativistic species				
$N_{\rm eff}$	0.2	0.045	-	Green 2019
Neutrino mass[c]				
$\sum m_\nu$ (eV, $\sigma(\tau) = 0.01$)	0.1	0.03	✓	Dvorkin 2019
$\sum m_\nu$ (eV, $\sigma(\tau) = 0.002$)		0.015	✓	
Accelerated expansion				
$\sigma_8(z = 1 - 2)$	7%	1%	✓	Slosar 2019
Galaxy evolution				
$\eta_{\rm feedback}$	50–100%	2%	✓	Battaglia 2019
$p_{\rm nt}$	50–100%	4%	✓	Battaglia 2019
Reionization				
Δz	1.4	0.3	-	Alvarez 2019
τ	0.007	0.0035	-	Alvarez 2019
Cluster catalog	4000	33,000	✓	
AGN catalog	2000	96,000	-	
Galactic science				
Molecular cloud B-fields	10s	> 860	-	SO 2022 GS
$\sigma(\beta_{\rm dust})$	0.02	0.005	-	SO 2022 GS
Solar System Science				
Distance limit for 5 $M_\oplus$ Planet 9	500 AU	900 AU	✓	Fienga 2020
Asteroid detections		$\sim 10,000$		
Transient detection distance				
Long GRBs, on-axis		1300 Mpc	-	
Low-luminosity GRBs		70–210 Mpc	-	
TDEs, on-axis		670 Mpc	-	

1.5 Sources of (LAT) Pointing Error

- Azimuth, Elevation, Corotator encoder offsets
- Axis misalignment between corotator, mirrors, elevation space and the LATR
- Physical elevation sag of mirrors with gravity
- Tilts of the entire platform
- Thermal fluctuations
- Distortions in focal plane from off-axis mirror design

(**?**) (**?**)

2 Simulating Time-Ordered Data

Simulation of the TOD from a telescope scan is done in TOAST. TOAST works by generating data from a simulated scan of a sky. Adjustments to either the TOD or the telescope properties (such as pointing) is done via apply operators. To simulate pointing errors I need to create a new pointing error operator, scan a simulated CMB map, then proceed with parameter estimation.

Before I can implement pointing errors, I need a way to qualify, and quantify, the efficacy of my operator. While I could scan a basic simulated CMB map, I am not confident I would be able to tell if my operator is doing exactly what I want it to do. Instead I want to scan a blank map with a single point source. In this way, the effects of my pointing errors will be more evident. To create this map I will use `healpy`.

I also need a way of recreating maps from the TOD. Both `TOAST` and `sotodlib` have methods for this.

3 Updates

2026-09-30

I have a simple simulation pipeline that can make a toast simulation scanning a custom map, with or without pointing jitter. Then run a ml mapmaker or a filter bin mapmaker on the resulting TOD.

The input map looks like this - for a 10arcmin source:

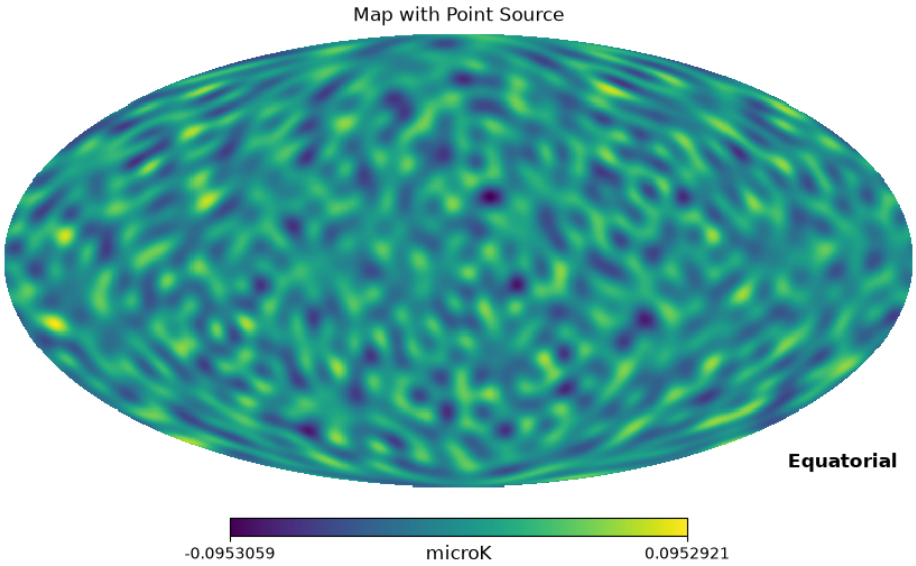


Figure 1: Input map for the simulation.

The toast sim also produces some pointing visualizations to confirm that the pointing jitter is being applied as expected.

This can be viewed in 3D:

After the ml mapmaker, the resulting map look like:

This work has been a good proof of concept for implementing a pointing jitter, however, there are major limitations to this setup for simulating a realistic LAT scan. First, this telescope/focal plane objects are not representative of a realistic LAT. Second, other elements of the simulation like noise, atmosphere, etc. are not included in a realistic manner. It would see that the best way to proceed is to learn how the realistic LAT sims are actually run and incorporate my pointing jitter into that framework.

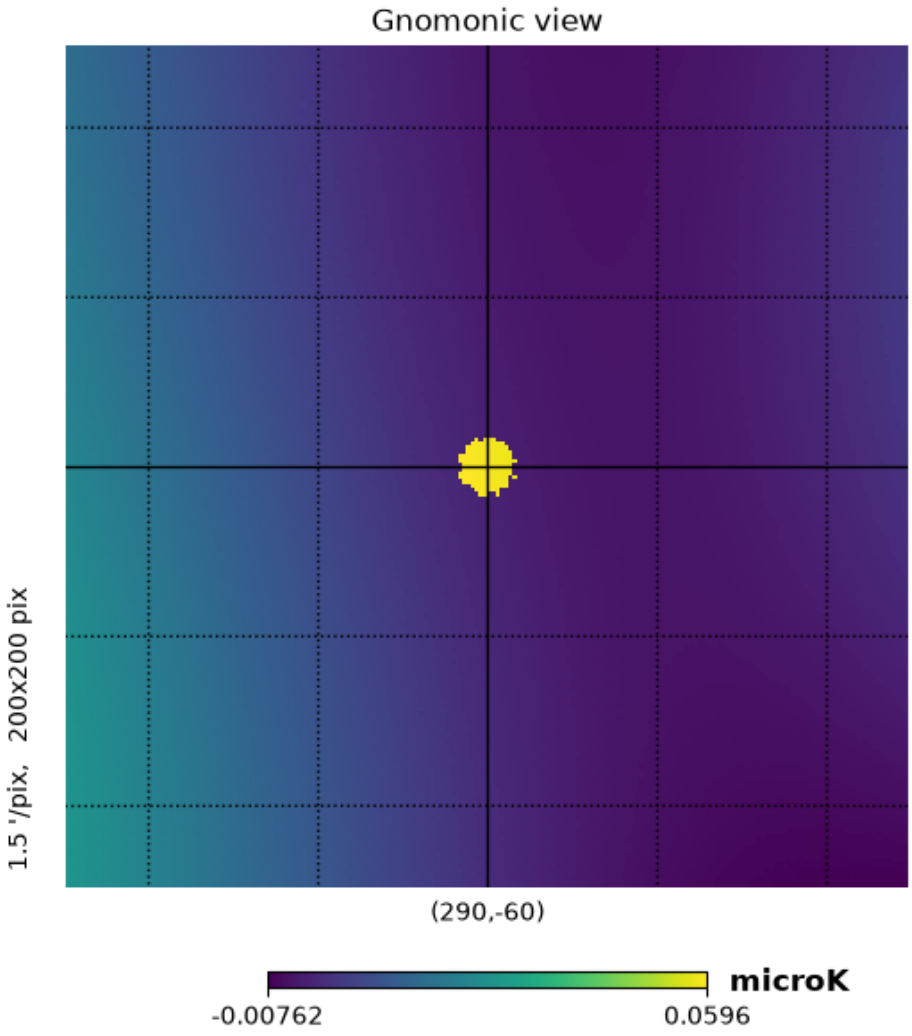


Figure 2: Input map for the simulation - gnomonic projection of the source.

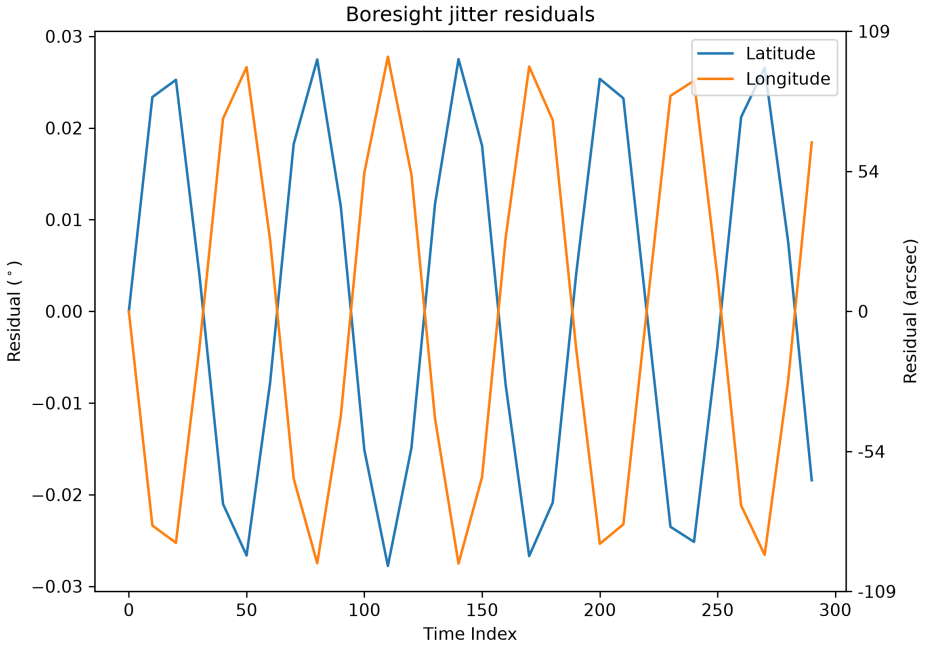


Figure 3: Pointing jitter boresight residuals.

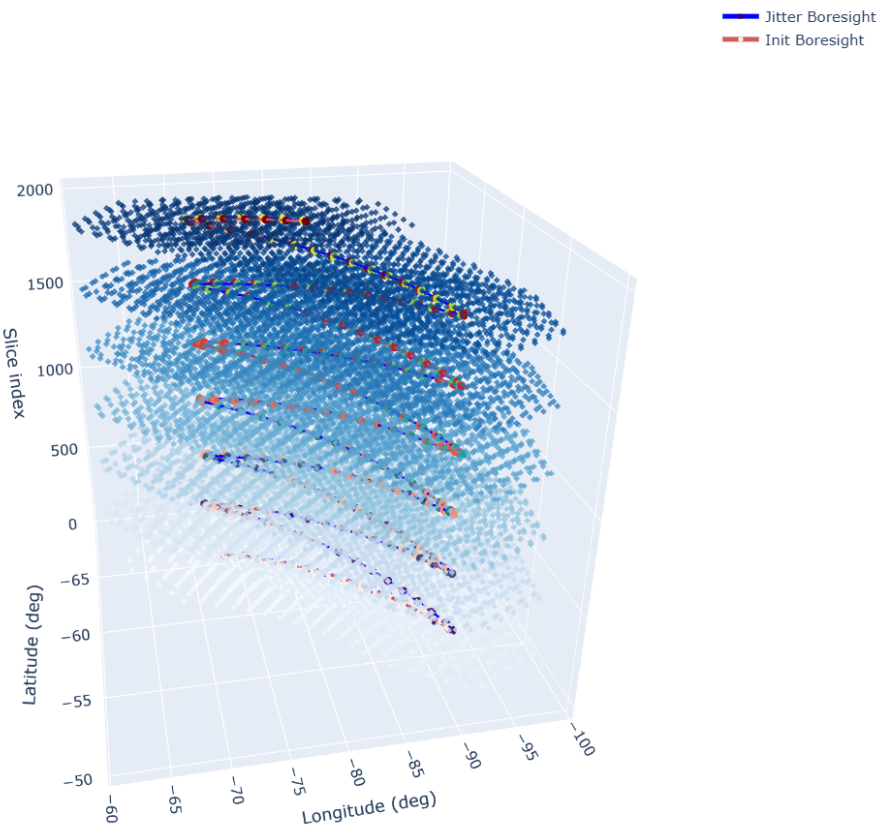


Figure 4: Pointing jitter boresight residuals.

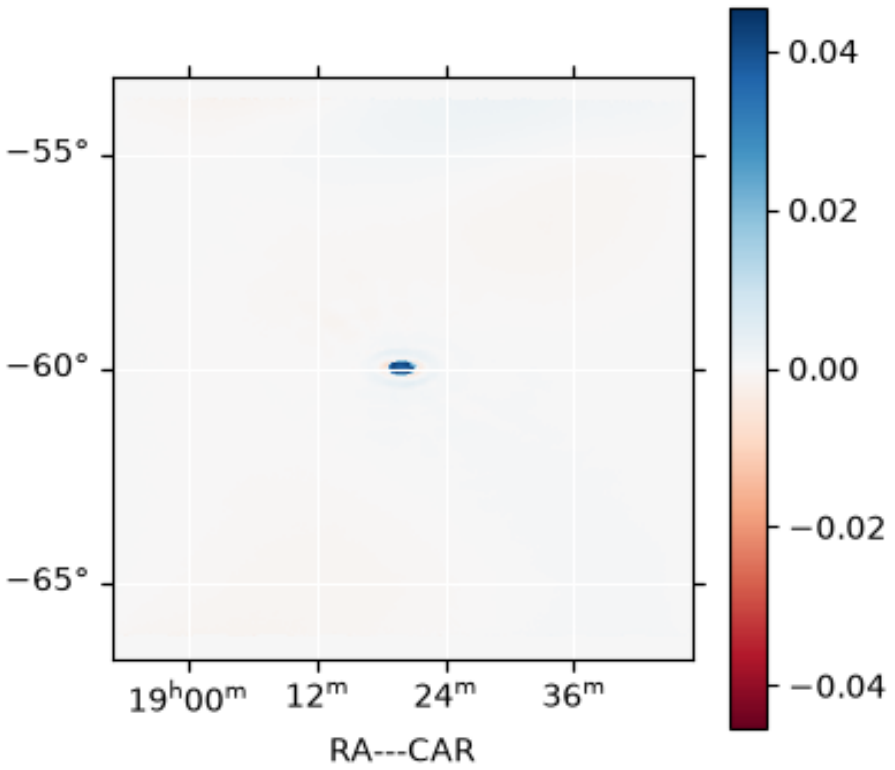


Figure 5: Output I map of ML mapmaker with no jitter.

2026-10-11

I have spent the better part of the last 10 days trying to figure out how to run a more realistic LAT toast sim. Here is what I have learned:

1. A realistic SO TOAST sim is largely run through the use of the command line interface (CLI) from the sotodlib module (`toast_so_sim`). The CLI has a huge (thousands) of possible command line arguments to specify which operators to apply and operator parameters. This interface can also likely take config files to ensure reproducibility but I haven't really played around with that.

2. While adding the new pointing jitter operator to the list of possible operators to call in the CLI is fairly straightforward, it is not clear how/where to include calling the new operator in the pipeline of a toast sim.

3. The large number of arguments are not really accompanied by clear instructions indicating which operators need to be specified for a particular type of simulation. For instance, I have been trying to scan a solar system object (SSO) for a semi-realistic simulation of a point-source. Turns out defining the SSO in your schedule does not make toast actually simulate the source, to do this, you need to enable the `--sim_sso.enable` argument. But this argument seems to also require other arguments to be specified. What those arguments are, and what they should be set to is not clear (for example, what beam file should I specify).

4. Running from a CLI makes many interactive things like plotting the impact of my operator much more difficult.

Before I can do anything, I need to make sure that running the toast sim CLI actually pull from my custom code (since my code is not yet incorporated into the greater TOAST, sotodlib packages). The instructions for doing this are mostly documented on the SO wiki page for using NERSC. In brief, the steps are:

1. Create a custom conda environment.

2. Install the sotodlib and toast packages from source.

Once this is done, the new operator must be added to the list of available operators for `toast_so_sim`

Because I did not want to deal with trying to figure out how to use the toast SSO method, I just had toast spit out the RA/Dec of where Mars was from its schedule, then adjusted my input maps to have a point source at that location.

To test that the CLI method was working, I ran a toast sim of my fake 'Mars' map without any pointing jitter. It took me a minute to get the inputs correct so that it would actually scan my custom map, and also run a filter bin mapmaker on the resulting TOD. After some trial and error, I was able to get the following map.

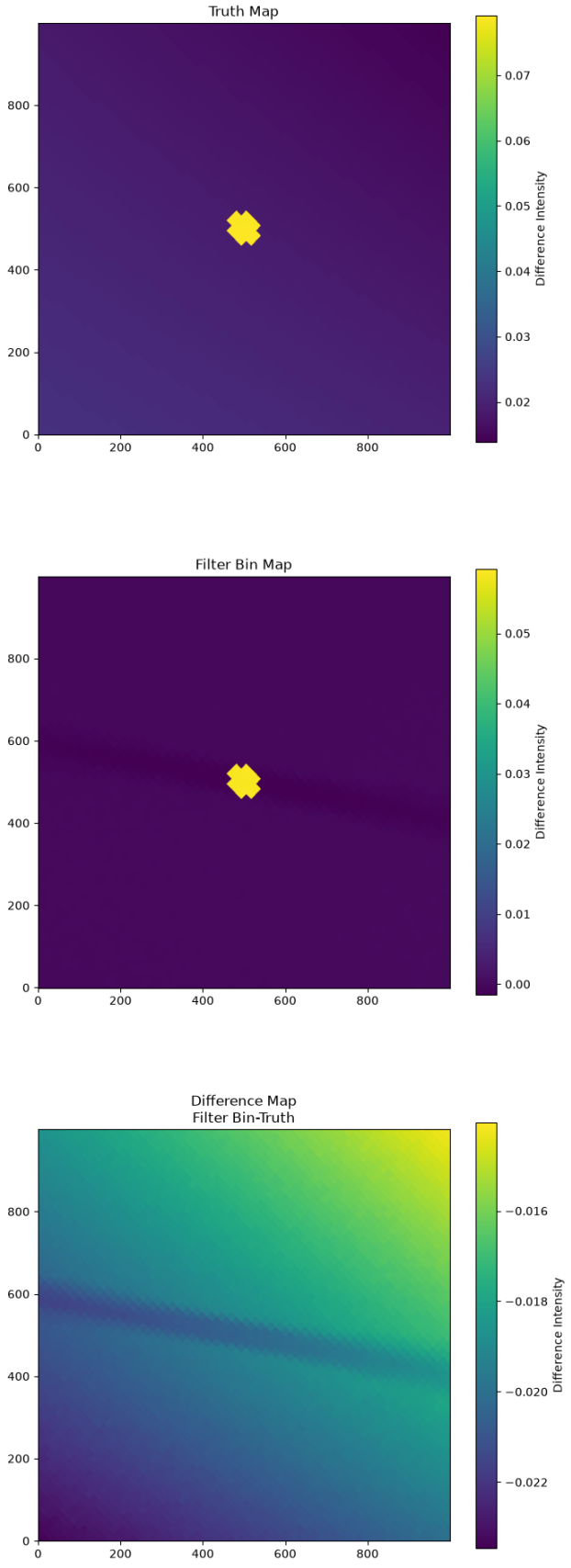


Figure 6: Difference between input map and filter bin scan.